UP-Gated Sale Token

How the "–" (Finish Up) step works, and how to discuss it in a FAANG / Amazon interview

1. The problem in one line

Salespeople could skip the UPS board and create a sale directly (even by calling the save API). That broke fairness in the rotation and made closing-ratio numbers wrong. **Goal: an employee can only create a sale if they really served a customer through the board.**

2. The flow in simple words

1. **Take a UP (+).** The salesperson is moved to "With Customers" on the board.
2. **Press "–" (Finish Up).** The server removes them from the working queue and opens a **finish log** row (`ups_finish_logs`, `ended_at = NULL`). They are now in "Finishing Up Paperwork". They are **not** put back in the waiting line yet.
3. **Answer "How did the customer find us / who helped?".** Saved to `customer_acquisition`. The outcome buttons stay locked until the server confirms this save.
4. **Choose one outcome:** Create Sale, Quotesheet, RTS (return), Capture Details (lead), or No Sale.
5. **Only Create Sale or Quotesheet asks for a token.** The browser calls `POST /ups/sale-token`. The server checks three things itself (it never trusts the browser):
   - the user is an **employee** (role read from the database),
   - they have an **open finish log**, and
   - acquisition was saved **after** that finish log started.

   If all pass, the server creates a random secret (32 random bytes → 64-char hex) and stores it in `sale_authorization_tokens`, linked to that finish log, with an expiry.
6. **The sales form opens.** A frontend route guard lets employees into the form only if they hold a token (or it is a phone order / RTS). This is just for user experience; the real check is on the server.
7. **"Save Your Sale".** The browser sends the ticket + `userId` + `saleToken` to `POST /api/invoice/save`. The server **spends** the token in one atomic SQL statement:

   ```
   UPDATE sale_authorization_tokens
      SET used = 1, used_at = NOW(), consumed_slug = ?
    WHERE token = ? AND user_id = ? AND store_name = ?
      AND used = 0 AND expires_at > NOW();
   -- 1 row changed  -> allowed, write the sale
   -- 0 rows changed -> 403 "You must complete a UP before creating a sale"
   ```
8. **If writing the sale fails** after the token was spent, the server **releases** the token (sets it back to unused), so the salesperson can retry.
9. **Workflow complete.** `/ups/finish-up/end` closes the finish log and puts the salesperson back in the waiting line in the correct position.

> **Other outcomes (RTS, Capture Details, No Sale)** never create a token. They finish their own workflow and close the finish log. **Managers and admins** are exempt. **Phone orders (PH)** are exempt by design (the salesperson stays in the line).

Picture of the flow

```
[+ Take UP] -> With Customer
     |
[- Finish Up] -> finish log OPEN  (not in waiting line)
     |
[Acquisition saved]  (outcome buttons unlock)
     |
     +-- RTS / Capture Details / No Sale -> own workflow -> close finish log
     |
     +-- Create Sale / Quotesheet
            |
        POST /ups/sale-token  (role + open log + acquisition checked)
            |
        token row: used=0, expires_at, finish_log_id (UNIQUE)
            |
        POST /api/invoice/save  (atomic used 0 -> 1)
            |-- fail to write -> release token
            |
        finish-up/end -> back in waiting line
```

3. The key design rules (remember these 6)

Rule	How it is enforced
One token per customer served	`UNIQUE(finish_log_id)` on the token table. A second "mint" returns the same token instead of creating a new one.
A token is used only once	Atomic conditional UPDATE (`used = 0` in the WHERE). Even with 50 parallel requests, only one wins.
Never trust the client	Role, store, open finish log and acquisition are all read on the server from the database.
Retries are safe	If the same ticket (same `slug`) retries with a token it already spent, the server returns OK (idempotent), not 403.
No lost token, no half sale	Spend the token before writing; release it if the write fails (a compensating action).
Safe rollout	Feature flag `SALE_TOKEN_ENABLED` (kill switch). Shipped dark first, logs every mint / consume / deny with a reason, then enforcement was turned on.

4. Real production issue (a great interview story)

> **What happened:** A furniture sale took longer than the 30-minute token life. The token expired. On retry, the code tried to **insert** a new token, hit the UNIQUE key on `finish_log_id`, and refused. The salesperson was **stuck forever**: they could not save the sale, and the finish log never closed.
>
> **Root cause:** The schema said "one token per finish log forever", but the real business rule is "one *active* token per finish log, and one *completed sale* per finish log".
>
> **Fix:** (1) Re-mint became an **UPDATE in place** (extend expiry where `used = 0`), so it never re-opens a finished sale. (2) TTL was raised from 30 to 720 minutes, with a same-business-day soft limit (16h). (3)

> The frontend re-mints and retries **once** on a 403. (4) Tested with 50 concurrent requests on real MySQL: still exactly one sale per token.

5. Amazon interview: how to tell it (STAR, about 2 minutes)

S – Situation: In our retail sales app, employees could bypass the customer-rotation board and create sales directly, which hurt fairness and corrupted closing-ratio reports.

T – Task: Make sure a sale can only be created after a real customer interaction, without slowing down the sales floor or breaking phone orders and returns.

A – Action: I designed a server-side one-time authorization token. It is created only when the salesperson picks "Create Sale", after the server checks role, the open interaction, and customer-acquisition data. It is spent with an atomic conditional update when the sale is saved, and released if the save fails. I made retries idempotent, shipped it behind a kill switch with structured logs, and ran it silently before turning enforcement on.

R – Result: Direct-API bypass was closed; every committed sale maps to exactly one served customer. When a long sale exposed an expiry deadlock, I found the root cause (insert vs. update on a unique key) and fixed it permanently, with a concurrency test at 50 parallel requests.

Ownership **Dive Deep** **Insist on Highest Standards** **Invent & Simplify** **Learn & Be Curious**

6. Drill-down questions and strong answers

Q1. Why a database token and not a JWT?
A JWT is stateless, so it cannot be "used once" or revoked without a server-side store anyway. We needed single use, expiry, and a link to one specific interaction. A database row gives all three and is easy to audit.

Q2. How do you guarantee the token is used only once under concurrency?
One atomic conditional UPDATE: `... WHERE used = 0 AND expires_at > NOW()`. The database row lock means only one request changes the row; we check `affectedRows === 1`. No read-then-write, so no race (no TOCTOU).

Q3. Why not create the token when "–" is pressed?
At that moment the outcome is unknown. It would give a sale token to RTS, No Sale and leads too. We mint only on the Create Sale / Quotesheet choice, and the server re-checks that an open interaction exists.

Q4. What if the server spends the token and then the invoice write fails?
A compensating action releases the token (`used` back to 0). Invariant: no sale without a spent token, and no spent token without a sale. In a larger system this is the Saga pattern.

Q5. The client times out, but the sale was actually saved. It retries. What happens?
Idempotency: we store `consumed_slug`. Same token + same ticket id → return success. The invoice write is an upsert on the unique slug, so no duplicate sale.

Q6. How did you choose the TTL?
We first chose 30 minutes, which was wrong for furniture sales that can take hours. Now: 12-hour TTL, a 16-hour business-day soft limit, and a safe re-mint. Lesson: base the TTL on real workflow length, not on security defaults alone.

Q7. Couldn't a user just call the save API with a fake token?

A fake or used token is strictly rejected (403), with no fallback. Tokens are 256-bit random values. Store and role are read from the database, not from the request.

Q8. What is the weakest part of your design? (Be honest here.)
The app has no JWT/session auth; the user id comes in the request body. So the token proves "this user id served a customer", not "the caller is this user". With real auth, I would bind the token to the session. Also, the token is not bound to one specific ticket, so an employee's active token could be used on a different ticket of their own. This is capped at one sale per interaction, so the risk is small.

Q9. How did you roll it out safely?
Dark launch: tokens were minted but not enforced. Structured logs recorded every deny reason (e.g. `no_user_id` showed old frontend versions). There was a temporary auto-grant grace period, then strict enforcement. Rollback is a single flag.

Q10. How would this work at Amazon scale?
DynamoDB with a conditional write (`ConditionExpression: used = false`) and a native TTL attribute for cleanup. The partition key is the token, with a GSI on the interaction id. Emit CloudWatch metrics for mint / consume / deny counts, with alarms on deny spikes.

Q11. Refresh, new tab, or a different computer?
The token is kept in localStorage for the same device. The server is the source of truth: the open finish log and the token row. On another device, the app re-mints and gets the same token back (idempotent), because only one token can exist per finish log.

Q12. What would you improve next?
(1) Bind the token to the ticket id. (2) Close the finish log on the server when the sale commits (today the frontend closes it). (3) Add a nightly cleanup job for expired tokens. (4) Add an alarm when deny counts go above normal.

7. One-line summary to say out loud

> "When the salesperson picks Create Sale, the server checks they really served a customer and issues a one-time, expiring token tied to that interaction. Saving the sale spends it with one atomic update: exactly one sale per customer served, retries are safe, and the bypass is closed."